INEM · IA pour les systèmes embarqués · Réseaux de neurones

Comprendre le perceptron, du pixel à la prédiction

Reconnaître des chiffres en quatre étapes progressives

Travaux dirigés et initiation à la recherche · Roland Cédric TAYO

Résumé

Ce travail conduit du neurone simple au réseau multicouche. Les fonctions sont écrites à la main : lire les pixels, calculer une sortie, corriger les poids, puis tester. Chaque étape distingue les activités et les résultats à conserver. Une étude des poids quantifiés relie reconnaissance, mémoire et mesures.

Mots-clés : perceptron ; classification ; apprentissage ; généralisation ; expérimentation.

Objectifs et parcours

Comprendre comment des nombres entrent dans un neurone, comment ses poids changent pendant l'apprentissage et comment vérifier ce qu'il a appris.

LE PARCOURS

Étape et dossier de référence	Ce que l'on découvre
1. Généralisation `1-Generalisation`	Un neurone distingue 0 et 1 ; on perturbe ensuite les pixels pour tester sa résistance au bruit.
2. Deux classes `2-Separation-deux-classes`	La règle de Widrow–Hoff apprend à partir d'un score réel, au lieu de la seule décision 0 ou 1.
3. Dix classes `3-Separation-dix-classes`	Dix neurones donnent dix scores. Le score le plus grand désigne le chiffre prédit.
4. Multicouche `4-Perceptron-Multicouches`	Une couche cachée et la rétropropagation permettent d'aborder les chiffres manuscrits MNIST.

> **Progression.** Les quatre étapes sont au programme. Le petit calcul précède le réseau complet. Le bloc final compare les poids quantifiés du réseau MNIST ; les poids de secours permettent de le commencer si l'apprentissage n'est pas terminé.

RÈGLE DE TRAVAIL

Écrire les calculs avec des boucles, en Python ou C, sans bibliothèque matricielle ni IA. Commenter, expliquer et tester chaque fonction. Python est conseillé aux étapes 1 à 3 ; C facilite la suite embarquée à l'étape 4. Les mesures de langages figurent page 7.

Prérequis : fonctions, boucles, tableaux et fichiers ; sommes de produits. Les dérivées et leur enchaînement sont nécessaires à l'étape 4.

ÉTAPE 1 · 1-GENERALISATION

Un neurone distingue les chiffres 0 et 1

Question. Peut-il reconnaître deux motifs, puis leurs versions abîmées ?

zero.txt

```
......
.****.
.*..*.
.*..*.
.*..*.
.*..*.
.****.
......
0
```

un.txt

```
......
...*..
..**..
...*..
...*..
...*..
..***.
......
1
```

Chaque fichier comporte **8 lignes de 6 caractères**, puis la classe en 9e ligne.

. devient 0 ; * devient 1.

Lire ligne par ligne : on obtient 48 entrées x_j, avec j de 0 à 47.

ACTIVITÉ — CONSTRUIRE, APPRENDRE, TESTER

1. Créer les deux fichiers. Écrire les fonctions de lecture, de calcul du score, de décision et de mise à jour. Initialiser les poids w_j dans [−0,1 ; 0,1], avec une graine enregistrée (par exemple 42). Initialiser le biais **b=−0,5** et fixer le pas **η=0,01**.
2. Pour un exemple, calculer le score a, la prédiction ŷ et l'erreur e. Puis corriger les poids et le biais :

$$a = \Sigma_{j=0}^{47} w_j x_j + b \; ; \quad \hat{y} = 1 \text{ si } a \geq 0, \text{ sinon } 0.$$

$$e = d - \hat{y} \; ; \quad g_j = -e\, x_j \; ; \; g_b = -e \; ;$$

$$w_j \leftarrow w_j - \eta\, g_j \; ; \; b \leftarrow b - \eta\, g_b.$$

Lire la recette : d est la classe attendue ; ŷ est la classe prédite. Σ signifie additionner ; ← signifie remplacer par la nouvelle valeur. Le biais b décale la frontière de décision. Le seuil θ=0,5 initial correspond à b=−θ. La règle du perceptron ne dérive pas le seuil discontinu ; ce n'est pas la descente de la perte quadratique des étapes suivantes.

3. Une époque présente les deux motifs, une fois chacun, dans un ordre mélangé. À la fin, **sans modifier les poids**, recompter les erreurs sur les deux motifs. Arrêter à zéro erreur, ou après 1 000 époques en signalant la limite.
4. Pour p=0, 10, …, 100 % de bruit, copier le motif et inverser exactement n=arrondi(48p/100) pixels **distincts**, tirés au hasard. Tester 50 copies par chiffre et par p. Garder les mêmes copies pour l'étape 2 ; ne jamais apprendre sur ces copies.

RÉSULTATS À CONSERVER

- Une figure à deux panneaux : erreurs sur les deux motifs selon l'époque ; taux d'erreur selon le bruit, avec une courbe par chiffre.
- CSV, graine et poids appris. Taux d'erreur (%) = 100 × nombre de mauvaises réponses / nombre d'essais.
- Quatre phrases : que change l'apprentissage ? Que se passe-t-il avec le bruit ? Ces tests ne prouvent pas la reconnaissance de toutes les écritures.

Sur papier : pour x=[1,0], w=[0,2 ; −0,1], b=−0,5 et d=1, calculer une sortie puis une mise à jour. Ce mini-exemple a deux entrées, pas 48.

ÉTAPE 2 · 2-SEPARATION-DEUX-CLASSES

Changer la règle d'apprentissage : Widrow–Hoff

Question. Que change une correction fondée sur la valeur du score, plutôt que sur la seule décision ?

Même motif x → Score réel a → Gradient a − d → Correction des poids

CE QUI CHANGE

Le score reste $a=\Sigma_j w_j x_j+b$. À l'étape 1, la correction utilisait $d-\hat{y}$: une décision déjà correcte ne produisait plus de correction. Ici, le gradient est **a−d** : le score continue à se rapprocher de la cible numérique 0 ou 1.

$$\delta = a - d \; ; \quad w_j \leftarrow w_j - \eta\, \delta\, x_j \; ; \quad b \leftarrow b - \eta\, \delta.$$

Pour classer après calcul : $\hat{y} = 1$ si $a \geq 0{,}5$, sinon 0.

Le score réel n'est pas limité à [0 ; 1] et n'est pas une probabilité. Le seuil de décision vaut ici 0,5 parce que les cibles numériques valent 0 et 1. Il ne remplace pas le biais b.

ACTIVITÉ — MODIFIER UNE CHOSE ET COMPARER

1. Repartir des **poids initiaux** de l'étape 1, pas de ses poids appris. Conserver les motifs, le biais initial et $\eta=0{,}01$. Remplacer seulement la règle d'erreur et utiliser le seuil de décision indiqué ci-dessus.
2. Sur un petit exemple, calculer à la main la correction du perceptron, puis celle fondée sur a−d. Écrire ce qui diffère. Dans le programme, distinguer explicitement `score`, `prediction` et `target`.
3. Présenter les deux motifs à chaque époque. Après l'époque, calculer la perte L sur les deux motifs avec des poids figés. Arrêter si $L \leq 10^{-4}$, ou après 1 000 époques. Si elle explose ou devient non finie, vérifier la mise à jour ; refaire ensuite un essai à $\eta=0{,}001$ en le signalant.
4. Tester les **mêmes copies bruitées** qu'à l'étape 1, sans mise à jour. Comparer la robustesse des deux règles.

$$L = (1 / 2N)\, \Sigma_{k=1}^{N} (d_k - a_k)^2 \; ; \quad \text{ici } N=2.$$

N est le nombre de motifs évalués ; k désigne un motif ; d_k est sa cible et a_k son score. Le carré pénalise l'écart numérique. Le facteur ½ simplifie la dérivée. Une perte numérique et un taux de mauvaises classes sont deux mesures différentes.

RÉSULTATS À CONSERVER

- Une figure : perte L selon l'époque et comparaison des courbes d'erreur au bruit des deux méthodes. Nommer les axes et les courbes.
- Un tableau donnant, pour chaque méthode, les scores et décisions sur les deux motifs propres, puis l'erreur à 10 %, 30 % et 50 % de bruit.
- Quatre phrases : quelle règle fonctionne le mieux dans votre essai, pour quel chiffre et dans quelles limites ?

> **À examiner, pas à supposer.** Widrow–Hoff réduit une erreur quadratique. Il ne garantit ni un écart égal aux deux classes, ni des courbes de bruit symétriques, ni une meilleure robustesse dans tous les cas.

Stabilité locale. Avec le biais, utiliser $\bar{x}=[x,1]$. Pour une présentation, $\eta\|\bar{x}\|^2<2$ limite le pas de Widrow–Hoff ; $\|\bar{x}\|^2$ est la somme des carrés, donc le nombre de pixels actifs plus 1. Vérifier cette quantité sur les motifs. Ce contrôle ne garantit pas la robustesse au bruit.

ÉTAPE 3 · 3-SEPARATION-DIX-CLASSES

Passer de deux chiffres à dix chiffres

Question. Comment choisir une classe parmi dix, au lieu de répondre seulement 0 ou 1 ?

48 pixels → 10 neurones linéaires → 10 scores → Le plus grand gagne

ACTIVITÉ — ÉTENDRE LE PROGRAMME

1. Créer les dix fichiers : `zero.txt`, `un.txt`, `deux.txt`, `trois.txt`, `quatre.txt`, `cinq.txt`, `six.txt`, `sept.txt`, `huit.txt` et `neuf.txt`. Même format : 8 lignes de 6 pixels, puis la classe de 0 à 9.
2. Écrire le vecteur cible d de longueur 10 : une seule case vaut 1, celle du chiffre attendu ; les autres valent 0. Par exemple, pour le chiffre 2 : d=[0,0,1,0,0,0,0,0,0,0].
3. Créer W, tableau de **10 lignes et 48 colonnes**, et 10 biais b_i. W_{ij} relie le pixel j à la sortie i. Initialiser les poids comme avant et les biais à zéro ; fixer η=0,01.
4. À partir d'un motif, calculer les dix scores puis appliquer Widrow–Hoff à chaque sortie :

$$a_i = \Sigma_{j=0}^{47} W_{ij}x_j + b_i \; ; \quad \delta_i = a_i - d_i,$$
$$W_{ij} \leftarrow W_{ij} - \eta\, \delta_i x_j \; ; \quad b_i \leftarrow b_i - \eta\, \delta_i.$$

i va de 0 à 9 ; j de 0 à 47. Calculer les dix erreurs à partir des scores avant mise à jour. Pour prédire, choisir $\mathbf{argmax_i\ a_i}$: l'indice du score le plus grand. En cas d'égalité, prendre le plus petit indice. Ne pas chercher dix décisions binaires indépendantes.

5. Une époque parcourt les dix motifs dans un ordre mélangé. Calculer ensuite, sans apprendre, le nombre de classes correctes et $L=(1/2N)\Sigma_k\Sigma_i(d_{ki}-a_{ki})^2$, avec N=10. Arrêter si $L \leq 10^{-4}$, ou après 1 000 époques ; noter le résultat réellement obtenu.
6. Pour chaque chiffre, produire 50 variantes à 10 % de bruit, sur des copies séparées. Remplir une matrice de confusion 10×10 : **ligne = chiffre réel ; colonne = chiffre prédit**. Chaque ligne contient donc 50 essais.

RÉSULTATS À CONSERVER

- Une figure à deux panneaux : taux de reconnaissance sur les dix motifs selon l'époque ; matrice de confusion sur les 500 variantes bruitées.
- Un tableau des dix prédictions sur motifs propres et le taux global sur les variantes : 100 × bonnes réponses / 500.
- Quatre phrases : quels chiffres se confondent ? Dix motifs appris ne représentent pas toutes les façons d'écrire les chiffres.

Contrôle. Vous devez avoir 480 poids et 10 biais. Chaque motif donne exactement une classe prédite. Les poids restent figés pendant les évaluations.

ÉTAPE 4 · 4-PERCEPTRON-MULTICOUCHES

Une couche cachée pour les chiffres manuscrits

Question. Comment apprendre des transformations intermédiaires au lieu de relier directement chaque pixel aux sorties ? Les données et leur lecture sont décrites page suivante.

MNIST : 28 × 28 → 784 entrées → 100 neurones cachés → 10 sorties

ACTIVITÉ — SUIVRE LES CALCULS, PUIS PROGRAMMER

1. Afficher une image MNIST et son étiquette. Convertir chaque pixel p_j en $x_j = p_j/255$: les entrées sont entre 0 et 1. Créer la cible d de dix valeurs comme à l'étape 3.
2. Créer $W^{(1)}$ de taille 100×784, $W^{(2)}$ de taille 10×100 et les biais $b^{(1)}$, $b^{(2)}$. Initialiser les biais à zéro ; tirer chaque poids entre $-1/\sqrt{n}$ et $+1/\sqrt{n}$, où n est le nombre d'entrées de sa couche (784 ou 100). Garder la graine.
3. Calculer d'abord la couche cachée, puis les sorties, avec la sigmoïde σ :

$$\sigma(u) = 1/(1+\exp(-u)) \; ; \quad \sigma'(u) = \sigma(u)[1-\sigma(u)].$$
$$u_h = \Sigma_j W^{(1)}_{hj} x_j + b^{(1)}_h \; ; \quad z_h = \sigma(u_h),$$
$$v_i = \Sigma_h W^{(2)}_{ih} z_h + b^{(2)}_i \; ; \quad s_i = \sigma(v_i).$$

h désigne un neurone caché (0 à 99), j un pixel (0 à 783), i une sortie (0 à 9). z est une activation cachée ; s est la sortie. exp est l'exponentielle ; σ′ est la dérivée. Les exposants (1) et (2) désignent les couches, pas des puissances.

4. Pour la perte d'un exemple $L = \tfrac{1}{2}\Sigma_i (d_i - s_i)^2$, calculer les signaux de correction de la sortie, puis de la couche cachée :

$$\delta_i = (s_i - d_i) s_i (1 - s_i),$$
$$\delta_h = z_h (1 - z_h) \, \Sigma_i W^{(2)}_{ih} \delta_i.$$

5. Quand **tous les δ sont calculés avec les anciens poids**, mettre à jour : $W^{(2)}_{ih} \leftarrow W^{(2)}_{ih} - \eta \delta_i z_h$; $W^{(1)}_{hj} \leftarrow W^{(1)}_{hj} - \eta \delta_h x_j$. Soustraire ηδ au biais correspondant. δ est le gradient de L par rapport au potentiel du neurone. Tous les gradients précèdent toutes les mises à jour.
6. Vérifier d'abord sur papier un réseau réduit 2→2→2, puis la même sortie dans le programme. Entraîner selon la page 6. Les dix sigmoïdes sont des scores ; leur somme ne vaut pas nécessairement 1. Prédire avec $\text{argmax}_i s_i$, sans modifier les poids au test.

RÉSULTATS À CONSERVER

- Pour amorcer : une image avec son label, les dimensions vérifiées et la trace du petit calcul 2→2→2.
- Pour terminer : une figure des pertes train/validation selon l'époque ; un tableau de reconnaissance sur le test et dix exemples réel/prédit.
- Une explication : comment l'erreur de sortie corrige-t-elle aussi la couche cachée ? Aucun score de 96 % n'est imposé ou garanti.

Une notation de perte. Aux étapes 2 à 4 : $L = (1/2N)\Sigma_k \Sigma_i (r_{ki} - d_{ki})^2$. r est le score réel (a ou s), N le nombre d'exemples ; la somme sur i parcourt les sorties. Chaque mise à jour utilise N=1. L n'est pas un taux d'erreur. Pour une sortie, la MSE du projet suivant vaut 2L : son gradient contient donc un facteur 2.

FICHE PRATIQUE · DONNÉES, VÉRIFICATIONS ET REMISE

Lire MNIST et terminer sans ambiguïté

1. LES QUATRE FICHIERS

Le miroir de la CVDF propose les fichiers compressés et décrit le format IDX. Télécharger puis décompresser les quatre fichiers `.gz`. Si les données du dossier de travail sont déjà disponibles, vérifier leurs en-têtes avant utilisation.

Fichier décompressé	Contenu et en-tête à lire
`train-images-idx3-ubyte`	60 000 images 28×28 ; 16 octets d'en-tête.
`train-labels-idx1-ubyte`	60 000 labels de 0 à 9 ; 8 octets d'en-tête.
`t10k-images-idx3-ubyte`	10 000 images de test ; 16 octets d'en-tête.
`t10k-labels-idx1-ubyte`	10 000 labels de test ; 8 octets d'en-tête.

Les nombres d'en-tête sont des entiers de 32 bits en **big-endian** : l'octet de poids fort vient en premier. Lire type, nombre d'éléments et dimensions. Une image occupe ensuite 784 octets non signés. Pour l'indice r, lire l'image à 16+784r et le label à 8+r ; les indices commencent à 0. Ainsi, les indices d'entraînement vont de 0 à 59 999.

2. PREMIÈRE EXÉCUTION DE L'ÉTAPE 4

1. Prendre les images train d'indices 0 à 4 999 pour apprendre et 5 000 à 5 999 pour valider. Ces ensembles ne se recouvrent pas. Les fichiers officiels de test restent réservés à l'évaluation finale.
2. Faire un premier essai η=0,01, graine 42, cinq époques. Pour la campagne commune, le réglage fourni est η=0,3, cinq époques, graines 42, 43 et 44 : il a été choisi sur validation, avant le test. Utiliser une mise à jour par exemple. Mélanger les exemples train à chaque époque ; chronométrer la première. À chaque fin d'époque, calculer perte et taux de reconnaissance sur train et validation, **sans mise à jour**. Garder les poids de plus faible perte de validation.
3. Évaluer une fois ces poids sur les 10 000 images de test. Si vous commencez par 1 000 pour vérifier le programme, appeler ce résultat « test partiel, N=1 000 ». Ne pas ajuster les réglages à partir du test.

Ce démarrage réduit sert à comprendre et déboguer. L'utilisation d'un plus grand ensemble d'apprentissage est un prolongement, avec une nouvelle partition documentée. Une baisse de perte ne garantit pas à elle seule une bonne reconnaissance sur le test.

3. VÉRIFIER ET CONSERVER

Pour chaque étape : sources commentées, tests, réglages, graines, poids, CSV, figure et analyse. Distinguer ce qui est terminé de ce qui est amorcé. Le bloc mémoire/quantification et la remise sont décrits pages 7 à 9.

En cas de blocage : vérifier d'abord lecture et dimensions ; puis normalisation, signes et ordre des mises à jour. Revenir au petit calcul avant de relancer un entraînement.

Sujet initial : « Reconnaissance de caractères », Pierre Andry, Université de Cergy-Pontoise (`toDo.pdf`). Adaptation pédagogique : Roland Cédric TAYO.

Lectures : CVDF — MNIST et format IDX ; Stanford CS231n — règle de chaîne et rétropropagation. Ces lectures aident à comprendre ; elles ne remplacent pas vos calculs.

Secours. Le dossier étudiant à télécharger fournit fiches, données et poids pour les trois graines, avec SHA-256 et contrôles de sortie. Leur utilisation doit être déclarée. Elle permet la recherche, sans remplacer la compréhension des fonctions. Lire le format dans README.md.

BLOC FINAL · COMPRENDRE LA CONTRAINTE MATÉRIELLE

Faire tenir le réseau : où vont les nombres ?

Question. Un fichier de poids qui tient en stockage tient-il aussi dans la mémoire de travail ?

ACTIVITÉ — COMPTER AVANT D'OPTIMISER

1. Pour 784→100→10, compter séparément les matrices W et les biais b. Vérifier le total de 79 510 paramètres. Un float32 occupe 32 bits, soit 4 octets.
2. Calculer le volume si tous les paramètres étaient en int8. Pour notre expérience, quantifier **seulement les matrices** : les 110 biais restent en float32. Ajouter séparément les échelles : une globale, ou deux par couche, stockées en float32.
3. Compter les tampons d'une inférence : 784 entrées, 100 activations cachées et 10 sorties, en float32. On peut accumuler chaque potentiel dans un scalaire, sans garder un autre vecteur. Préciser que pile, code, runtime et données du système s'ajoutent.
4. Comparer ces comptes à l'hypothèse **Raspberry Pi Pico / RP2040 : 264 Kio de SRAM et 2 Mio de Flash externe**. Ce n'est pas le Raspberry Pi Linux du projet final. Aucune carte supplémentaire n'est demandée. Les capacités viennent de la documentation, pas d'une mesure de ce TD.

À distinguer. La Flash conserve le programme et les constantes. La RAM contient ce qui doit être modifié pendant le travail. Copier tous les poids en RAM et les lire depuis la Flash ne demandent pas le même budget. Ne pas conclure qu'un modèle est impossible à partir de sa seule taille.

RÉSULTATS À CONSERVER

Un tableau : paramètres, payload des matrices, biais, échelles, tampons. Nommer les hypothèses de placement. Écrire « calculé » pour un compte analytique et « mesuré » uniquement pour une observation réelle.

QUEL LANGAGE CHOISIR ?

Étapes 1–3 : Python est conseillé pour les petits calculs et les figures ; C convient aussi. Étape 4 : C rapproche de l'inférence sur carte. Python pur reste possible. Sur le PC de validation, 5 000 images et cinq époques à η=0,01 ont donné :

Implémentation	Apprentissage seul	Évaluations train/val
C · float32	0.671 s	2.052 s
Python pur · float64	45.052 s	27.822 s

Mesures tracées : Ryzen 5 7535HS, Linux WSL2, GCC 11.5.0 −O2, Python 3.12.3. Les deux versions sautent les pixels exactement nuls pendant l'apprentissage ; leurs précisions arithmétiques diffèrent. Chargement et écriture des poids exclus de ces deux colonnes. Ce ne sont ni des durées imposées aux étudiants ni des mesures de carte.

Fiche Pico ; RP2040, §2.6.2. 1 Kio=1 024 octets ; 1 Mio=1 024² octets.

PETITE RECHERCHE · UNE QUESTION, UN PROTOCOLE FIXE

À partir de combien de bits la reconnaissance se dégrade-t-elle ?

Comparer les mêmes poids sur images propres et bruitées. Le but est d'établir une limite dans ces conditions, pas de promettre un nombre de bits universel.

ACTIVITÉ — ÉCRIRE L'HYPOTHÈSE, PUIS QUANTIFIER

1. Remplir la fiche d'hypothèse et commiter **avant les résultats**. Le seuil commun de dégradation est une perte d'au moins **1 point de reconnaissance** par rapport au FP32 de la même graine et condition.
2. Garder 32 bits comme référence ; tester les poids sur 8, 4 et 2 bits. Les biais ne changent pas. Pour B bits, poser $Q=2^{B-1}-1$. Pour un groupe de poids, prendre $s=\max|w|/Q$. Si tous sont nuls, prendre $s=1$.

$$q=\text{borne}(\text{arrondi}(w/s), -Q, Q) ; \quad \hat{w}=sq.$$

B est le nombre de bits ; Q la plus grande valeur positive du code ; s l'échelle ; w le poids initial ; q son code ; $\hat{w}$ sa reconstruction. Arrondir au plus proche, à mi-distance en s'éloignant de zéro. **2 bits symétriques donnent ici trois niveaux −1,0,1** : un code reste inutilisé.

3. Faire une échelle **globale** commune à W1 et W2, puis une échelle **par couche**. Reconstruire les poids avant le calcul flottant. Ce TD simule une quantification des poids ; ce n'est pas un noyau entier accéléré.
4. Utiliser les trois modèles issus des graines 42,43,44. Geler les choix avant le test. Évaluer une fois chaque modèle/précision prévu sur les 10 000 images propres et sur leurs copies bruitées. À 32 bits, une seule référence par graine suffit.
5. Le bruit MNIST commun inverse **78 pixels distincts** par image : $p \leftarrow 255-p$. C'est 10 % demandé, arrondi sur 784 pixels. La graine du bruit est 2026 ; le même masque est réutilisé pour toutes les précisions et graines d'apprentissage. Le générateur fourni fixe ces copies.

RÉSULTATS À CONSERVER

CSV : `seed,bits,scale_mode,acc_clean,acc_noisy,n`. Les accuracies sont des pourcentages ; n est le nombre d'images par condition. Tracer reconnaissance selon les bits, avec toutes les graines, deux conditions et deux modes d'échelle. Comparer au FP32 de la même graine. Signaler les écarts défavorables, nuls ou non monotones.

Conclusion. Quel premier niveau testé franchit le seuil ? Est-ce vrai pour chaque graine, chaque échelle et chaque condition ? Expliquer les arrondis et les limites : un réseau, MNIST, un type de bruit et du calcul flottant. Ne pas ajuster un nouveau modèle sur ces résultats de test ; une confirmation indépendante serait nécessaire.

MESURER · VERSIONNER · EXPLIQUER

Transformer les résultats en preuves relançables

ACTIVITÉ — CHRONOMÉTRER SANS INVENTER UN GAIN

1. Vérifier les sorties et le rechargement. Employer le même C et les mêmes options −O2 pour la référence et les variantes. Noter CPU, OS, compilateur, options, charge ; fréquence et température si elles sont accessibles.
2. Sous Linux, utiliser `clock_gettime(CLOCK_MONOTONIC)`. Échauffer 100 appels. Doubler K depuis 1 jusqu'à au moins 100 ms sur le lot de référence, avec un plafond de 2^{24}. Garder le même K pour une paire.
3. Faire trois campagnes de dix paires, en inversant l'ordre référence/variante à chaque paire. Préparer les entrées avant le chronométrage, les varier de façon déterministe ; conserver un checksum utilisé après le lot. Fichiers, affichage et allocations restent hors du temps mesuré.
4. Garder durée totale, K et checksum dans le CSV. Durée par appel = durée totale/K. Rapporter médiane et min–max, en précisant le périmètre. La quantification reconstruite utilise encore du FP32 : **un gain de temps n'est pas attendu par principe**.

Portée. K est le nombre d'appels ; le checksum est une somme de contrôle des sorties. Ces mesures décrivent le PC. Raspberry Pi et microcontrôleur : **non mesurés**. Énergie : **non mesurée**. Le projet suivant mesurera réellement le Pi.

ACTIVITÉ — CONSERVER L'EXPÉRIENCE

Créer `src/ tests/ configs/ deployment/ artifacts/ experiments/ figures/` et `.github/workflows/`. Inviter **rolln7drktayau**. Un commit cohérent par opération, puis `git push`. Nommer les jalons : `day1-data`, `day1-core`, `day1-trained`, `day1-quantization`, `day1-submission`. Faire relire les tests par un autre membre.

Le manifeste indique commit, SHA-256 des données/poids/binaires, configuration, machine, compilateur, options, K et unité. La CI exécute les tests ; ses temps ne représentent pas une carte. Le squelette fourni est à compléter avec vos contrôles du modèle.

RÉSULTATS À CONSERVER

Dépôt relançable ; fiches de calcul et de prévision ; tests ; hypothèse préalable ; graines et poids identifiés ; CSV bruts ; courbes ; conclusion limitée ; déclaration d'IA et bilan individuel. Pour une présentation, expliquer une fonction, un résultat, une limite et le prochain test. Indiquer clairement les étapes terminées ou amorcées et les poids de secours utilisés.

IA autorisée. Déclarer les aides substantielles. Montrer une sortie d'IA corrigée et le test qui prouve la correction. Chaque membre doit prédire puis expliquer une petite modification. La compréhension et les preuves comptent ; la quantité de code ne suffit pas.

Règles. Bibliothèques standard et outils de figures autorisés. Aucun NumPy, BLAS, framework ou autodifférentiation pour score, activation, perte, gradient, entraînement ou inférence. Le banc numérique ne donne pas la rétropropagation. Sources : MNIST/CVDF ; Stanford, règle de chaîne. Calculs du cours actuel : diapositives 11–15 ; mémoire/quantification : 23–26 ; enquête : 34–36.